LoopFuzz: Closed-Loop Runtime Admission Control for LLM-Guided Stateful Protocol Fuzzing

Ketang Chen[a], Xiaolei Ren[a,*]

[a]*Macau University of Science and Technology, Avenida Wai Long, Taipa, Macau, China*

Abstract

Stateful protocol fuzzing is difficult because a syntactically valid message can still violate the current session state, especially when a large language model (LLM) can inject candidates into a long-running campaign. We present LoopFuzz, a closed-loop runtime admission-control design for LLM-guided stateful protocol fuzzing. LoopFuzz treats model outputs as bounded hypotheses rather than queue entries. It promotes request-producing outputs only when runtime evidence supports parseability, response-grounded acceptability, state reachability, and downstream gain; scheduling-only outputs remain bounded hints. Failed executions become counterexamples for local repair, and state-aware scheduling triggers model assistance only under adaptive plateau conditions. On nine primary AFLNet/ChatAFL-lineage targets, archived repeated-run summaries show the clearest evidence for improved response-derived IPSM exploration: after correction, edge improvements remain significant on eight targets versus AFLNet and six targets versus ChatAFL. Branch-coverage improvements are also significant on seven targets versus AFLNet and four targets versus ChatAFL, but Forked-daapd remains a negative primary case. Auxiliary NSFuzz branch artifacts exceed LoopFuzz on five of eight nominal shared targets, while the Mosquitto MBFuzzer archive is available only as a worker-level branch-count artifact. Together, the data indicate improved state exploration in the closest implementation lineage and branch-coverage gains on most primary targets, with mixed coverage outcomes against specialized state-aware artifacts.

Keywords: Protocol fuzzing, Large language models, Runtime admission control, Stateful testing, Counterexample-guided local repair

1. Introduction

Network protocol implementations remain difficult security targets because correctness depends on interaction history rather than isolated inputs [1–5]. An FTP server, for example, rejects a well-formed `PASS` command if no valid `USER` command has established the login state. Unlike file fuzzing, protocol fuzzing must preserve message syntax, temporal order, and request-response progress across a multi-step session [5–10].

Traditional coverage-guided fuzzers work well for stateless parsers. They fit network servers poorly when later behavior depends on earlier valid exchanges [11–15]. Stateful greybox fuzzers such as AFLNet and StateAFL partially close this gap. They infer progress from responses and replay valid traces [1–4, 7, 16, 17]. Even so, byte-level mutation still struggles to synthesize the context-sensitive messages needed to cross deep protocol boundaries.

Large language models offer a plausible source of such messages. ChatAFL and later LLM-assisted fuzzers show that a model can extract structure from specifications, logs, or documentation and produce fluent candidates [18–25]. But fluency is not enough. A request can be grammatically plausible and still be invalid for the live session state. Such executions spend budget without improving queue quality.

LoopFuzz places a runtime admission controller between LLM generation and queue admission. It treats the LLM as a hypothesis generator rather than an oracle. It derives message hypotheses from available RFC fragments, seed traces, and observed responses. It stores failed executions as counterexamples and uses them for counterexample-guided local repair rather than unconstrained regeneration. It also uses inferred protocol state machine (IPSM) feedback to prioritize frontier states and rare transitions, and to trigger assistance only when ordinary mutation stalls. The name *LoopFuzz* emphasizes closed-loop admission and local repair, avoiding any suggestion of formal verification of protocol semantics, model behavior, or server correctness.

The contributions of this paper are as follows:

- **Hypothesis-Driven Modeling.** LoopFuzz derives message grammars and field constraints from RFC fragments, seed traces, and observed responses, and maintains them as explicit runtime hypotheses.
- **Runtime Admission Boundary.** LoopFuzz inserts a runtime admission controller between the LLM

*Corresponding author: Xiaolei Ren. Email address: xlren@must.edu.mo

Email addresses: 3250006913@student.must.edu.mo (Ketang Chen), xlren@must.edu.mo (Xiaolei Ren)

and the queue. It checks structure before trial execution and uses bounded execution evidence to evaluate response acceptability, state reachability, and downstream gain before durable queue promotion.

- **Counterexample-Guided Local Repair.** LoopFuzz preserves failed request-response traces as counterexamples and uses them to patch one field constraint or production at a time, which constrains model freedom through local, auditable repairs.
- **State-Aware Scheduling.** LoopFuzz leverages IPSM topology, state productivity, and adaptive plateau detection to prioritize underexplored frontier states and to request model assistance only when mutation alone stalls.
- **Finding on Closed-Loop Control.** The repeated-run archive shows that, within the AFLNet/ChatAFL lineage, LoopFuzz primarily improves response-derived state exploration and improves branch coverage on most primary targets; specialized state-aware external artifacts give a mixed coverage picture.

Accordingly, the paper evaluates LoopFuzz as a campaign-control policy, not as a standalone message generator. In stateful protocol fuzzing, generation matters only when it survives runtime evidence and improves downstream queue quality.

2. Background and Motivation

Table 1 contrasts LoopFuzz with representative fuzzers. ChatAFL-style systems use protocol and interaction context to guide LLM generation. The distinction studied here is whether generated proposals are mediated by an explicit runtime admission boundary before they influence a long-running stateful campaign.

2.1. Coverage-Guided Fuzzing vs. Network Targets

A coverage-guided fuzzer operates through a repeated feedback cycle of seed selection, mutation, execution, coverage measurement, and novelty-based queue update [12–15]. This loop is effective for file parsers, but network targets impose three constraints that matter here:

1. **Temporal state dependence:** server behavior depends on the order and semantics of earlier messages, not only on the current payload [1–4].
2. **Strict message format:** protocols require delimiters, field layouts, and often checksum or length consistency, so many mutated messages die before they reach deeper logic [4, 26, 27].
3. **Execution instability:** sockets, forking daemons, and asynchronous response handling complicate both coverage collection and campaign throughput [17].

2.2. Stateful Greybox Fuzzing (SGF)

SGF addresses this gap by treating the target through an *inferred protocol state machine* (IPSM). In the AFLNet lineage, this machine is an approximate response-derived state machine, not a deterministic model of the protocol semantics. We use a Mealy-style abstraction $M = (Q, I, O, \widehat{\Delta}, \widehat{\Lambda}, q_0)$ to describe the scheduling signal, where:

- Q is the set of response-derived state identifiers;
- I is the set of client messages;
- O is the set of observed server responses;
- $\widehat{\Delta}$ records observed transitions between response-derived states under concrete executions;
- $\widehat{\Lambda}$ associates those observed transitions with response codes, response texts, or adapter-level response summaries; and
- q_0 is the initial connection state.

AFLNet-style fuzzers infer Q and transition evidence from observed responses and use that structure to bias later exploration toward rare or promising parts of the protocol. The abstraction can be partial and history-dependent: asynchronous replies, races, timeouts, daemon state, and environmental dependencies can make the same message sequence expose different response evidence across runs. Here, the IPSM matters as a scheduling and progress signal. It gives LoopFuzz a notion of response-level exploration that is richer than raw edge coverage alone, but it is not treated as a complete or deterministic protocol semantics model.

2.3. LLM Integration into Protocol Fuzzing

Large language models make it easier to propose structure-aware protocol messages from RFC text, interaction history, or observed responses. That helps with syntax and message diversity, but it does not solve the harder control problem: a fluent request can still be wrong for the live session state.

2.4. A Motivating Example: Semantic Drift

Consider an FTP fuzzing campaign against ProFTPD, as illustrated in Fig. 1. The fuzzer executes `USER` and `PASS`. It then queries the LLM for the next request. The model returns `RETR target`. The request is syntactically fluent but semantically premature. The client lacks an active data channel via `PASV` or `PORT`. The server rejects it with `425 Use PORT or PASV first`.

We call this failure *semantic drift*. The request is locally plausible, but it violates the precondition of the current session state. An open-loop LLM path may execute and retain the request because it looks fluent, leaving later mutation biased toward a rejected branch instead of the missing state transition. LoopFuzz handles this drift through runtime validation rather than through a purely pre-execution oracle.

Table 1: Among the listed systems, LoopFuzz combines LLM hypotheses with runtime admission and online repair; representative prior fuzzers differ in input prior, state signal, and admission boundary.

System	Input prior	State signal	Admission rule	Online repair
Coverage-guided mutation fuzzing [11, 12]	Byte mutation	None	None	No
AFLNet [1]	PCAP mutation	Response	Novelty	No
StateAFL [2]	PCAP mutation	Memory	Novelty	No
ChatAFL [18]	Context-guided LLM generation	Interaction history	No runtime admission boundary	No
LoopFuzz (ours)	**LLM hypotheses**	**IPSM frontier + validation**	**Runtime admission**	**Yes**

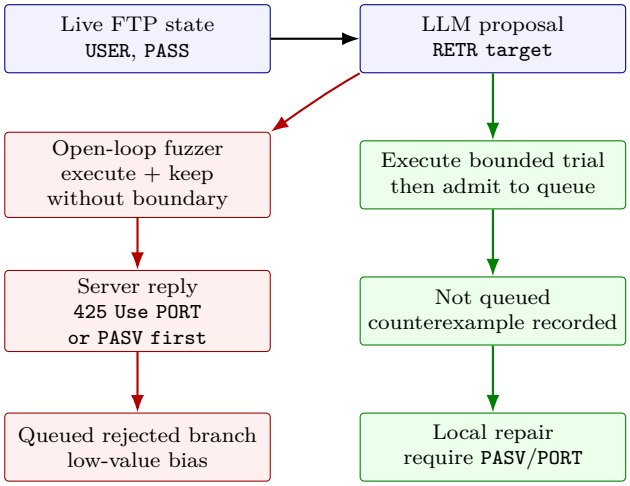

Figure 1: Semantic drift in an FTP session. The model proposes `RETR` before the session opens a data channel. Open-loop fuzzing can execute and retain the rejected branch. LoopFuzz does not decide $U/R/G$ before execution: after P permits a bounded trial, the `425` response makes the candidate fail post-trial admission. The candidate is not queued; the trace is recorded as a counterexample for repairing the missing `PASV/PORT` precondition.

The controller first checks local structure and parseability, then performs a bounded trial execution whose `425` reply becomes negative evidence. The candidate is not promoted into the long-lived queue, and the failed trace is recorded as a counterexample. The local repair step then patches the missing `PASV/PORT` precondition instead of regenerating the full continuation. This example motivates the design claim: LLM assistance should be mediated by runtime evidence rather than by surface fluency alone.

3. Related Work

We position LoopFuzz against the prior lines that are closest in mechanism and evaluation.

Earlier structure-aware fuzzing used explicit structure, inferred snippets, token constraints, or grammar guidance to preserve validity [26–32]. These systems improve input quality, but they do not address the control problem that arises when an external model proposes state-dependent protocol messages.

Stateful greybox and state-aware network-service fuzzing address protocol interaction more directly. AFLNet, StateAFL, NSFuzz, and the broader SGF line infer or expose state structure from server responses, process memory, or program variables [1–3, 16, 33]. They then use that structure to prioritize rare or productive traces. These systems define the broader comparison landscape, but they do not all expose the same state abstraction, instrumentation path, or archived endpoint metrics.

Later systems extend this idea to packet-sequence exploration, format-sensitive protocol fuzzing, and communication-heavy environments [4–8, 34–38]. LoopFuzz asks a narrower question: given an AFLNet-style response-derived IPSM and the ChatAFL-style LLM proposal path, does an integrated runtime-admission strategy make the campaign more reliable? AFLNet and ChatAFL are therefore the closest implementation-lineage baselines. StateAFL, SGF, NSFuzz, Bleem, Logos, and MBFuzzer remain important external comparison points; their different state abstractions, instrumentation paths, and artifact semantics motivate the separate diagnostic tables in the evaluation.

Adjacent work studies state-aware or interaction-heavy fuzzing in robotic policies, firmware, Linux drivers, middleware, hardware, and embedded systems [39–45]. Although not all are protocol fuzzers, they reinforce the point that state feedback can dominate raw mutation quality.

Target-aware scheduling and semantic validation form another relevant line. Directed greybox fuzzing, seed selection, and strategy adaptation study how to steer budget toward high-value behaviors [11–15, 46, 47]. Semantic validation and fuzz-driver generation show that stronger inputs often require an explicit acceptance interface [48, 49]. LoopFuzz applies that lesson to LLM proposals.

Our repair loop also borrows the broad counterexample-driven intuition from counterexample-guided abstraction refinement (CEGAR) in formal verification [50]. The analogy is limited: LoopFuzz records failed executions as concrete evidence for local message-hypothesis patches, but it does not construct a formal abstraction, prove refinement soundness or completeness, or compute minimal counterexamples.

Work on dependency inference, data-flow guidance, and learned signals outside protocol fuzzing also supports treating guidance as bounded evidence rather than as an oracle [51–53].

LLM guidance addresses the synthesis bottleneck more directly. ChatAFL performs context-guided LLM generation from RFCs and interaction histories [18];

adjacent systems use LLMs for general input generation, documentation-guided constraints, device fuzzing, or fuzz-driver generation [19–23, 54]. Our work is closest to ChatAFL, and differs by adding an explicit runtime admission boundary: LoopFuzz asks whether a proposal should be admitted, whether its hypothesis survives runtime evidence, and how the IPSM should guide the next query.

Benchmarking practice is also part of the context. ProFuzzBench established a reusable baseline for repeated protocol-fuzzing experiments [16]. Recent methodology work shows how easily comparisons become unstable without fixed budgets, repeated trials, and careful interpretation of coverage signals [55–57]. We follow that guidance because runtime-validated assistance is especially sensitive to campaign history.

4. LoopFuzz Design

Fig. 2 summarizes the three coupled loops in LoopFuzz: the Grammar Hypothesis Loop, the Fuzz Execution Loop, and the LLM Intervention Loop.

LoopFuzz preserves the stateful greybox fuzzing execution path. It sets a strict control boundary between LLM generation and durable queue admission. A model output can affect the campaign through two surfaces. Request-producing outputs change the durable queue only after runtime evidence supports admission. Non-request control actions, such as target-state redirection or seed prioritization, are treated as bounded scheduling hints after schema and identifier checks; their value is still mediated by later executions and AFLNet-style novelty logic. The admission rule for request candidates has two stages: structural and local parseability checks run before a candidate is tried, and response behavior, inferred state reachability, and downstream campaign value are evaluated after a bounded trial execution. The method has four coupled components: grammar hypothesis generation, runtime admission control, counterexample-guided local repair, and state-aware scheduling.

This control boundary is the core idea of the paper. ChatAFL provides context-guided LLM generation from protocol and interaction context, but it does not impose a separate runtime admission boundary before an extracted suggestion can affect the campaign. LoopFuzz adds that boundary: IPSM growth, state productivity, and hypothesis fitness determine when the LLM is queried, what context it receives, and whether a tried proposal is promoted into the campaign.

The three loops in Fig. 2 define the analytical roles of the method. The Grammar Hypothesis Loop constructs and repairs message hypotheses from RFC knowledge, seed traces, and observed responses. The Fuzz Execution Loop performs state selection, mutation, bounded execution, IPSM update, and novelty checking. A periodic monitor summarizes edge-growth behavior for plateau detection; it is not a per-execution LLM trigger. The LLM Intervention Loop is activated only under plateau conditions and returns either a validated continuation or a bounded control action.

The remaining labels in Fig. 2 denote control interfaces rather than independent claims. Tier-1 sampling checks parsed execution regions, Tier-2 repair patches counterexamples, schema validation checks model replies, and bounded action dispatch maps a validated request-producing proposal into a trial. Only request candidates that pass the post-trial admission predicates become durable queue entries. Scheduling-only actions are schema- and identifier-checked control hints rather than admitted inputs. The prompt reuse filter and query isolation bound repeated or failed model calls.

4.1. Grammar Hypothesis Loop

Let $\mathcal{L}(c, s, \Sigma)$ denote the LLM generation function under prompt context c, state summary s, and protocol knowledge Σ. LoopFuzz uses $\mathcal{L}$ in two stages. During initialization, it preserves ChatAFL's template extraction and seed enrichment path. Initial enrichment is scheduled as a bounded phase so that corpus diversification does not dominate early campaign time. During fuzzing, LoopFuzz adds a lazily initialized hypothesis layer. This layer is activated at the first plateau event and updated through sampled validation and local repair. The separation is important. The initialization phase broadens the initial seed set. Runtime hypotheses support runtime-validated control during long campaigns.

The Grammar Hypothesis Loop draws on three evidence sources when they are available: RFC fragments, queued seed traces or packet examples, and observed server responses, including response codes and error texts. For each message type m, the loop maintains a hypothesis $h_m = (G_m, \Phi_m)$. Here, G_m is an ABNF-style or template-style production, and Φ_m is a set of field constraints. In LoopFuzz, Φ_m captures length bounds, enumerated values, token dependencies, and simple cross-field relations. This representation is more constrained than free-form text and lighter than a full formal protocol specification. We choose it because sampled online validation must remain cheap.

The evidence sources need not be complete. RFC fragments supply structure, seed traces provide executable examples, and observed responses provide target-specific evidence about rejection, progress, and state change. LoopFuzz therefore requires testable hypotheses, not a full formal protocol specification; runtime admission decides whether those hypotheses can affect the campaign.

4.2. Runtime Admission Controller

The runtime admission controller is the central mechanism in our design. It makes queue-admission decisions for request-producing model outputs through two validation stages. Stage 1 checks whether a model proposal is structurally well formed and locally parseable before it is sent. Stage 2 performs a bounded trial execution and then evaluates the observed response, inferred state movement, and

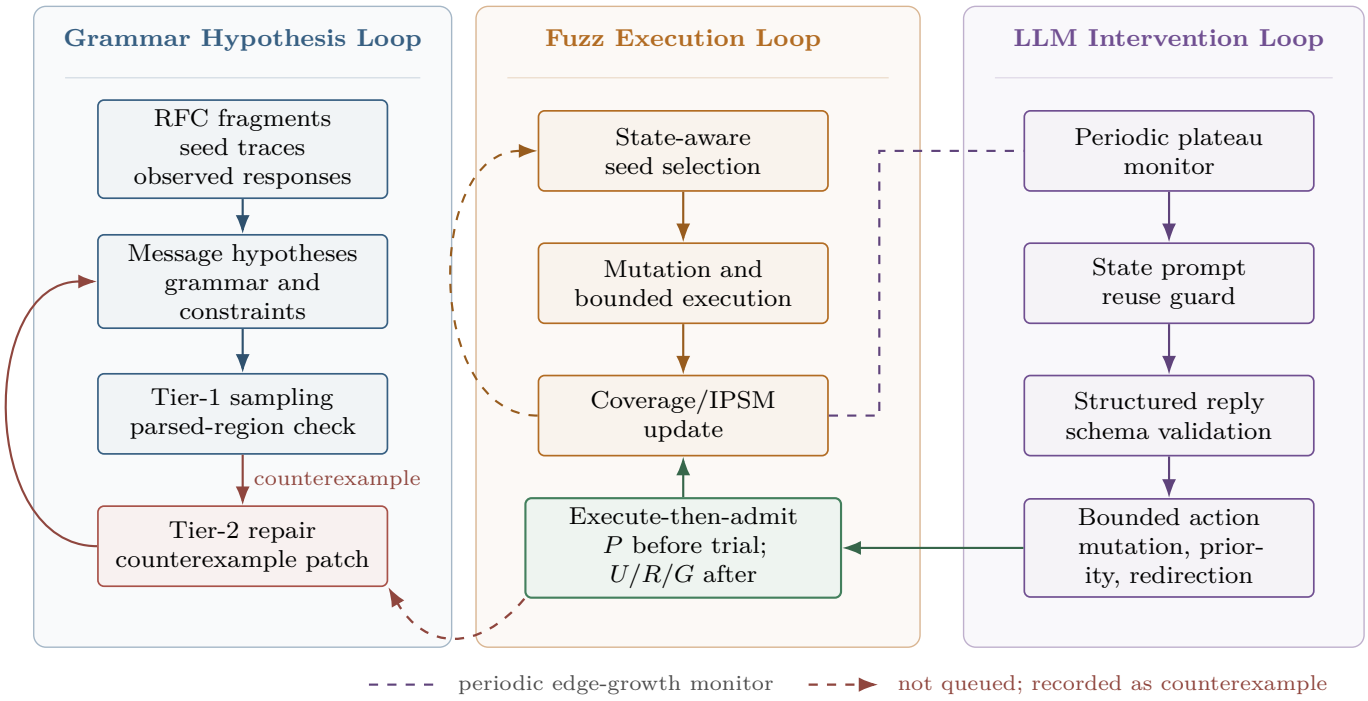

Figure 2: LoopFuzz changes the LLM from a queue producer into a bounded policy proposer. The Grammar Hypothesis Loop maintains message hypotheses, the Fuzz Execution Loop updates coverage and IPSM feedback, and the LLM Intervention Loop proposes bounded interventions. The plateau gate is driven by a periodic edge-growth monitor rather than by every single execution. Runtime admission is execute-then-admit: structural checks run before a bounded trial, while response, state, and coverage checks decide afterward whether the proposal is queued or recorded as a counterexample.

coverage or IPSM gain. A request candidate x is retained only if

$$A(x) = P(x) \wedge U(x) \wedge R(x) \wedge G(x),$$

where P denotes pre-trial local parseability, while U, R, and G are post-trial predicates computed from the bounded execution: U denotes response-grounded acceptability, R denotes evidence of new or redirected state reachability, and G denotes downstream gain in IPSM structure or coverage. If P fails, the proposal is not executed. If P passes but any post-trial predicate fails, the trial trace can be recorded as a counterexample or used to update state productivity, but the candidate is not promoted as a durable queue entry. Control actions that do not create a request, such as target-state redirection or seed prioritization, are outside this formula: they undergo schema and identifier checks, alter only scheduling metadata, and are evaluated indirectly by subsequent executions.

Table 2 gives the operational definition used to make this rule reproducible. The table separates the pre-trial structural gate from the post-trial evidence gates and lists the minimum event fields needed to replay an admission decision. These are the event-level fields that a complete artifact should preserve; as discussed in Section 8, the current archive does not contain normalized admission-event logs from which cross-target admission-rate summaries can be reconstructed.

The distinction between R and G is intentional. R asks whether the candidate can reach a meaningful protocol state under the response-derived IPSM. G asks whether that execution produced measurable campaign novelty. A trial can therefore pass U and R yet fail G when it reaches an already-known state without new coverage, IPSM, or queue value.

The cleanest ablation for this design would be *LoopFuzz-no-admission*: it would keep the same model endpoint, prompts, query budget, plateau triggers, and scheduling policy, but skip the $P/U/R/G$ checks and directly enqueue request-producing model suggestions while retaining the same schema checks for control actions. The current archive does not contain that condition, so the experiments evaluate LoopFuzz as an integrated control policy.

The FTP example in Fig. 1 illustrates the same two-stage logic. The candidate `RETR target` can satisfy local syntax under P, so LoopFuzz may try it in a bounded validation execution. The live `425` response then makes U fail because no data channel has been negotiated. The trial also provides no useful new transition or coverage signal, so R and G do not support admission. LoopFuzz therefore declines to promote the candidate into the queue and records a counterexample that constrains local repair of the missing `PASV`/`PORT` precondition.

4.2.1. Structured Output and Parseability

The plateau stage accepts either a concrete next-request candidate or a bounded structured-action object. Runtime validation first checks structural well-formedness, field

Table 2: Operational definition of the $P/U/R/G$ admission predicates for request-producing candidates. P is checked before execution. U, R, and G are checked after the bounded trial; post-trial failures may update counters or repair evidence but do not promote the candidate.

Pred.	Timing and input	Pass rule	Fail rule	Audit fields and effect
P				
Parse.	Pre-trial. Schema-checked reply, cleaned request or mutation-producing action, and current hypothesis $h_m = (G_m, \Phi_m)$.	Supported action type; required fields; valid field bounds; delimiter or terminator consistency; local parse under h_m.	Malformed JSON/action, unknown action, missing terminator, type/length violation, or parser failure.	Event id, action/message type, schema and parse flags, parse error, hypothesis id. No execution; failure stops the candidate.
U				
Accept.	Post-trial. Live response bytes, extracted code/state, timeout/disconnect status, and adapter hints.	Response is not classified as a rejection sink; adapter or productivity evidence may rescue an apparent error state.	Default text-protocol rejection: 4xx/5xx, timeout, reset, unparsable reply, or explicit `ERROR/BAD/invalid/denied`.	Response code/class, timeout flag, error hint, productivity, rejection reason. Failure blocks promotion and may create repair evidence.
R				
Reach.	Post-trial. Response-derived state sequence, source state, scheduled target/frontier state, and current IPSM.	Reaches a non-rejection state, reaches the scheduled target, escapes a repeated rejection sink, or creates a new response-derived relation.	Stays in the same rejection/error sink or yields no usable state sequence.	Source/destination states, state sequence, target flag, new relation, state productivity. Updates IPSM/frontier evidence; admission still requires G.
G				
Gain	Post-trial. Coverage delta, IPSM novelty, queue-favored status, and pre-trial campaign state.	Adds coverage bits/branches, a new IPSM node/path/edge, or a favored corpus entry.	Acceptable and reachable, but repeats a saturated state with no coverage, IPSM, or queue value.	New bits, branch delta, IPSM novelty, favored flag, gain reason, promotion flag. Passing all predicates promotes the candidate.

types, length bounds, delimiter or terminator consistency, and action-schema consistency. LoopFuzz then tests local parseability against the current message hypotheses. It reuses parsed message regions already available during ordinary execution. This first stage can reject structurally invalid interventions before any trial execution. It does not decide whether a syntactically valid request is useful; that decision depends on the post-trial predicates in Table 2.

4.2.2. Response-Grounded Acceptability

Parseability alone is insufficient. A parsed request can still be semantically premature. LoopFuzz therefore classifies the response observed during the bounded trial execution and treats a non-rejection response as positive evidence. For response-code protocols, 4xx/5xx states are the default rejection class; timeouts, early disconnects, unparsable replies, and explicit error texts are also negative evidence. The classification is conservative rather than absolute: a target adapter or accumulated productivity evidence can keep an apparent error state from becoming a permanent dead end if it repeatedly leads to new paths. Rejected or persistently unproductive responses reduce state productivity and hypothesis fitness. This gives an execution-grounded notion of acceptability rather than a semantic proof. The controller asks a narrow post-trial question: did live target behavior support promoting this candidate into the campaign?

4.2.3. State Reachability and Coverage Gain

The final admission question is whether the trial helped the campaign. LoopFuzz separates the answer into a state predicate and a novelty predicate. Reachability R is computed from the response-derived state sequence: did the candidate reach a non-rejection state, the scheduled target state, or a new state relation? Gain G is stricter and queue-facing: did that execution produce new coverage, a new IPSM node/path/edge, or a favored corpus entry? This separation handles the common case in which a candidate is state-valid but already saturated. The controller therefore uses IPSM-derived reachability and AFLNet-style novelty signals, not text quality alone. Fluent but repeatedly unproductive candidates may still consume a bounded validation execution, but they are filtered out before durable queue promotion instead of accumulating as low-value queue bias.

4.3. Counterexample-Guided Local Repair

When sampled validation fails repeatedly, LoopFuzz records a concrete counterexample. The record contains the failure-triggering request fragment, the observed response, and the mismatch against the current hypothesis. Repair is deliberately local. Instead of rewriting the full protocol description, the repair interface constrains each update to one field constraint or one production fragment. This single-patch discipline narrows the model's search space, limits unconstrained edits, and makes the repair path auditable. The resulting update is reintegrated into protocol matching and re-evaluated by later sampled validation.

This repair boundary is intentionally weaker than formal grammar repair. The one-field or one-production constraint is enforced through the repair prompt and audit log, not through hard AST differencing. The current implementation also stores concrete failed traces without counterex-

ample minimization. We therefore claim bounded local repair, not complete protocol inference.

4.4. State-Aware Scheduling and Plateau-Triggered Assistance

Reducing malformed generations solves only half of the problem. Stateful fuzzers also stagnate when the campaign keeps revisiting locally dense IPSM regions while failing to cross narrow protocol bridges. LoopFuzz therefore treats IPSM feedback as a first-class scheduling signal.

To address this, LoopFuzz augments state selection with a topology-and-productivity bonus $\beta(s)$. The policy deliberately separates low out-degree from useful low out-degree, so that response-derived dead ends are not rewarded only because $\deg^+(s)$ is small:

$$\beta(s) = \beta_{\text{frontier}}(s) \cdot \rho(s)$$

Let $n_s = \textit{selected\_times}(s)$ and $p_s = \textit{productivity}(s)$. The frontier term and penalty term are:

$$\beta_{\text{frontier}}(s) = \begin{cases} 4.0 & \text{if } s \notin Q \\ 4.0 & \text{if } \deg^+(s) \le 1 \\ 2.0 & \text{if } \deg^+(s) \le 3 \\ 1.0 & \text{otherwise} \end{cases}$$

$$\rho(s) = \begin{cases} 0.5 & \text{if } h(s) = 1 \\ 0.7 & \text{if } h(s) = 0,\ n_s \ge 30,\ p_s < 0.005 \\ 1.0 & \text{otherwise.} \end{cases}$$

Here $h(s) \in \{0, 1, 2\}$ is the online error-productivity hint: 0 means unknown, 1 means likely error or rejection, and 2 means confirmed productive. The implementation first assigns a topology bonus from the response-derived IPSM and then applies an acceptability/productivity penalty. Thus an absent or low-out-degree state can receive the maximum frontier term, but a response-derived rejection state is halved and an unknown state that remains unproductive after 30 selections is softened by a 0.7 multiplier. Confirmed productive states keep the topology bonus. This matches the reported artifact: terminal or rejection-heavy states are not excluded solely by degree, but repeated rejection or low productivity reduces their scheduling weight.

The reported implementation uses the constants above as a control policy, not as a claim of global optimality. The hint is initialized from response-code classification and updated online, so states can be promoted or demoted as evidence accumulates rather than fixed by an early response pattern. The error-productivity penalties and the $(30, 0.005)$ gate reduce repeated investment in response-derived rejection sinks while allowing recovery if later executions become productive. The 64-call intervention budget prevents unbounded LLM overhead. We report these constants for auditability, not as a full sensitivity analysis.

A complementary graph stagnation check exists only in the MQTT adaptor. When an MQTT run sees more than 50 consecutive rounds without IPSM-node growth, the scheduler uses random state selection with 40% probability to diversify broker exploration. This path is disabled for the nine non-MQTT primary targets, so the primary lineage results do not depend on node-stagnation randomization.

Plateau detection uses the IPSM edge growth rate $\dot{E}$, measured as edges gained per minute. LoopFuzz recalculates this rate every 60 seconds of wall-clock time and uses it to set an adaptive threshold:

$$\tau(\dot{E}) = \begin{cases} 700 & \text{if } \dot{E} > 5 \\ 600 & \text{if } 1 < \dot{E} \le 5 \\ 512 & \text{if } \dot{E} \le 1 \end{cases}$$

The initial value is τ=512, aligned with the ChatAFL plateau floor. Textual and MQTT runs use the same adaptive range in the reported implementation; MQTT can additionally pin the initial floor through a target-specific configuration option, but the selected Mosquitto archive is not used for performance conclusions. More generally, LoopFuzz treats plateau triggering as a rate-sensitive control decision, not as a fixed-frequency query rule.

Assistance is invoked only when edge growth remains weak and the model-intervention budget is not exhausted. In the reported configuration, the plateau path opens when the unproductive streak reaches τ and prior model interventions remain below 64. Once inside that path, hypotheses are initialized lazily on first use. Local repair remains gated by validation count and accumulated counterexamples. The prompt reuse filter covers the most recent 64 plateau-context hashes rather than model outputs. This prevents assistance from reacting to the same state summary again and again.

The plateau prompt exposes a compact state summary rather than a raw execution log. It includes IPSM node and edge counts, the current edge-growth rate, pending favored seeds, queued paths, the current target state, and the least productive states. The admissible action space is also bounded. A model can redirect the target state, reprioritize a small set of seeds, or propose bounded mutations over an existing seed. Request-producing actions must pass the runtime admission boundary before queue promotion. Scheduling-only actions must pass schema and identifier checks and then affect only state or seed selection; their usefulness is tested by subsequent executions. This design keeps the LLM in the role of a state-directed policy proposer rather than an oracle.

MQTT-specific scheduling is treated as a protocol adaptor, not as the general mechanism of LoopFuzz. For MQTT campaigns, packet-type selection uses a state-indexed action policy, and mutation-family selection uses a bandit-style controller. These signals help expose broker forwarding behavior to the IPSM, but they are protocol-adaptor details rather than primary performance-evaluation evidence. In short, assistance is conditional, budgeted, state-directed, and scoped by protocol evidence.

LoopFuzz also supports protocol-adaptor signals that are not headline evidence. For MQTT, the adaptor trans-

Algorithm 1: Runtime-Validated Plateau Recovery in LoopFuzz

Input: Target T, LLM interface $\mathcal{L}$, corpus C, edge-growth signal $\dot{E}$
Output: Coverage bitmap $\mathcal{B}_{global}$ and IPSM $\mathcal{M}$

Initialize $\mathcal{B}_{global}$ and $\mathcal{M}$
while *the campaign budget remains* **do**
  $s \leftarrow \textsc{SelectSeed}(C, \mathcal{M})$
  $(trace, Q) \leftarrow \textsc{MutateExecute}(T, s)$
  Update $\mathcal{B}_{global}$, $\mathcal{M}$, and frontier scores from $(trace, Q)$
  if *sampled validation is due* **then**
    Validate active hypotheses on parsed regions of Q
  end
  if *the adaptive plateau gate opens* **then**
    Initialize hypotheses on first use, if needed
    if *counterexample evidence opens the repair gate* **then**
      Patch the lowest-fitness hypothesis locally
    end
    $c \leftarrow \textsc{BuildPrompt}(\mathcal{M}, C, Q, \dot{E})$
    if c *is not in the recent prompt cache* **then**
      $r \leftarrow \mathcal{L}(c)$ under process isolation
      Execute each parsed request candidate as a bounded trial if P holds
      Queue the request only if post-trial U, R, and G also hold
      Apply scheduling-only actions only after schema and identifier checks
      Record failed post-trial request candidates as counterexamples or productivity evidence
    end
  end
end

lates between binary packets and textual hypotheses, drives broker interactions, and records differential behavior across roles. A lightweight oracle can log authentication bypass, state-order violations, information leaks, traversal patterns, injection patterns, or denial-of-service indicators. These signals support auditing and triage, but they are not treated as confirmed vulnerabilities because crash deduplication, root-cause analysis, exploitability review, and oracle calibration are not normalized.

Algorithm 1 specifies the plateau path and the hypothesis-repair gate. It formalizes how state-aware triggering, runtime admission checks, and counterexample-guided local repair interact within one campaign loop.

This integration imposes a bounded operating model. Initial enrichment is separated from long-running admission control, and hypothesis construction is deferred until the first plateau event. Model queries use process isolation, and network cleanup is bounded to limit persistent stalls on forking daemons. These mechanisms are operating conditions rather than separate novelty claims.

5. Experimental Evaluation Design

We organize the evaluation around three research questions that match the paper's main claims. The main experiment is a lineage experiment: it asks whether the integrated LoopFuzz control strategy changes a campaign that already has response-derived IPSM feedback and ChatAFL-style LLM proposals. Auxiliary NSFuzz artifact counts and policy perturbations provide additional context for branch coverage, state exploration, and component sensitivity.

- **RQ1 (Whole-Campaign Effect):** Does the integrated LoopFuzz strategy avoid the coverage regressions that can arise from direct open-loop generation?
- **RQ2 (IPSM Escape Dynamics):** Does the evaluated integrated policy, with frontier scheduling and plateau-triggered intervention, sustain IPSM growth after the AFLNet/ChatAFL baselines stall?
- **RQ3 (End-of-Campaign Outcome):** Under equal 24-hour budgets in the AFLNet/ChatAFL lineage, how does LoopFuzz affect final `b_abs` and final response-derived IPSM `edges`?

5.1. Experimental Setup

We ran all experiments on a dedicated Ubuntu 20.04 server with 32 CPU cores and 64GB RAM. The benchmark suite spanned 10 implementations across seven protocol families:

1. **FTP:** LightFTP, bftpd, ProFTPD, and Pure-FTPd.
2. **SMTP:** Exim.
3. **RTSP:** Live555.
4. **SIP:** Kamailio.
5. **DAAP/DACP:** Forked-daapd.
6. **HTTP:** Lighttpd1.
7. **MQTT:** Mosquitto.

The benchmark suite contains 10 targets, but the provided archives support different comparisons on different subsets. The primary comparison uses AFLNet, ChatAFL, and LoopFuzz on the nine off-the-shelf non-MQTT targets: LightFTP, bftpd, ProFTPD, Pure-FTPd, Exim, Live555, Kamailio, Forked-daapd, and Lighttpd1. This primary comparison is selected because AFLNet supplies the response-derived IPSM lineage and ChatAFL supplies the direct LLM-assisted predecessor that LoopFuzz modifies. The endpoint and trajectory results are computed from the primary archive `ten_groups_data_ten`. The policy-ablation table is computed from a separate `ten_groups_ablation_ten` archive. The two archives share targets and selection rules, but their LoopFuzz "full" rows are independent repeated-run cells rather than the same run set.

The equal-budget primary subjects are textual request/response protocols, matching the setting where LoopFuzz's parsers, response classifiers, and IPSM feedback are available.

Both ChatAFL and LoopFuzz used the same pinned OpenAI-compatible `gpt-5.4-mini` chat endpoint. The call configuration was fixed by call class. Legacy ChatAFL template extraction and seed enrichment use temperature 0.5

with at most five attempts. LoopFuzz hypothesis generation and counterexample-guided local repair use temperature 0.3 with at most three attempts. Plateau assistance uses temperature 1.2 with at most two attempts, because this path asks for exploratory scheduling actions rather than deterministic schema repair. The optional MQTT conformance check uses temperature 0.2 with one attempt and is not part of the primary performance table. All calls set `max_tokens`=4096 and do not set a model-side random seed, `top_p`, presence penalty, or frequency penalty; unspecified sampling parameters therefore follow the provider defaults.

The transport and failure policy is also fixed. Each API request uses a 30-second connection timeout, a 120-second total request timeout, and a low-speed abort if throughput remains below one byte per second for 60 seconds. Failed non-authentication requests sleep for two seconds before the next attempt; authentication failures abort the retry loop. The plateau path additionally runs the model call in a forked helper process and kills that helper after 60 seconds if no result is returned. Missing credentials, exhausted retries, malformed JSON, failed schema validation, or helper timeout produce no admitted model action; the fuzzing campaign continues with ordinary mutation and scheduling. System prompts are fixed per call class, while protocol name, seed examples, observed responses, and state summaries are supplied as user/context content. We did not hand-specialize prompt text per target. The primary table uses approximately 24-hour archived repeated runs under the deterministic selection rule described below, with no per-system cherry-picking of directories or best runs.

Because the plateau temperature is nonzero, the prompt reuse filter is not a deterministic memoization cache. It is a bounded repetition guard: the implementation hashes the concatenation of the seed examples, recent history, and state-context summary, keeps the most recent 64 hashes in a ring buffer, and skips a plateau call when the same stalled context reappears. Raw plateau replies and per-call prompt/completion token counts are retained in run-level audit records. This logging mitigates auditability loss from stochastic model calls, but it does not make the model path bit-for-bit reproducible.

Resource isolation and concurrency.. Each long-running sample is launched in an isolated ProfuzzBench-style container. In the standard archive, each container uses a one-CPU quota, an 8GB memory limit, an 8GB swap limit, and an initialization wrapper; the controller records the container identity, waits for completion, and recovers one compressed artifact per run. Runs are quota-limited but not pinned to physical cores.

Within a target-fuzzer invocation, simultaneous containers equal `NUM_CONTAINERS`; a 10-run cell can therefore run as 10 one-CPU containers or as smaller batches. The aggregate artifact preserves selected terminal summaries but not the global batch schedule, so we cannot reconstruct co-scheduling across the planned primary matrix. Endpoint comparisons therefore use completed approximately 24-hour runs selected by the same rule, and wall-clock curves are descriptive traces under the same container quota rather than CPU-time-normalized throughput claims.

Port conflicts are avoided by container network namespaces. Non-MQTT targets bind their services inside each container and pass a container-local endpoint to AFLNet-style network mode. These endpoints are not published to the host, so concurrent containers can reuse target-local network settings without cross-run conflicts. MQTT artifacts use separate container networks or generated worker endpoints, but the MQTT archive is excluded from performance conclusions: the LoopFuzz Mosquitto rows are much shorter than 24 hours, and the MBFuzzer archive exposes a different Mosquitto-only coverage surface without AFLNet-lineage `edges`.

Campaigns are bounded by a fixed wall-clock wrapper with a short kill grace and preserved exit status. Target reset is handled by the shared AFLNet-lineage harness and, where available, target-specific cleanup hooks. Offline coverage replay restarts or kills daemons with bounded cleanup commands before accumulating branch summaries. The launcher does not set a fixed AFL random seed, and the archive does not preserve per-run PRNG seeds. We therefore treat repeated runs as independent stochastic trials rather than bit-for-bit replays.

Run filtering and missing-data handling.. The intended primary matrix is 10 runs per fuzzer-target pair. We retain completed runs with at least 1400 archived minutes and final `b_abs`/IPSM `edges`. If more than 10 eligible rows remain, the deterministic rule keeps the lowest 10 run IDs before inspecting endpoint values, which avoids endpoint-based best-run selection. In the provided primary archive, all nine non-MQTT target-fuzzer cells contain 10 selected eligible runs. Table 3 reports the recoverable selected inventory. The ablation archive is filtered with the same rule; its full-controller column is used only as the local baseline for Table 10.

5.2. Baselines, Metrics, and Statistical Protocol

We used two baseline layers. The *primary* baselines were AFLNet as the response-derived stateful greybox baseline and ChatAFL as the direct LLM-assisted predecessor, which places LoopFuzz in the closest implementation lineage. The *auxiliary* reproduced baseline was NSFuzz [33] on eight non-HTTP/non-MQTT targets. A separate MBFuzzer archive exists for Mosquitto and contains worker-level coverage-replay branch counts. Because these auxiliary artifacts expose different metric surfaces from the AFLNet-lineage selected summaries, the evaluation reports them in separate diagnostic tables.

The main dependent variables are final `b_abs` and final `edges`. For AFLNet, ChatAFL, and LoopFuzz, the `b_abs` field is the absolute branch-coverage count from the same selected-summary and coverage-replay pipeline. LightFTP,

Table 3: Recoverable run-selection inventory for the primary AFLNet/ChatAFL-lineage comparison. Each fuzzer cell reports selected eligible runs over the planned 10-run primary matrix, followed by the number of planned rows not represented in the selected cache. The provided primary archive supplies 10 selected eligible runs for every non-MQTT target-fuzzer cell; the current package still does not preserve a complete raw pre-filter launch ledger.

Target	AFLNet	ChatAFL	LoopFuzz	Recoverable status
LightFTP	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
bftpd	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
ProFTPD	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Pure-FTPd	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Exim	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Live555	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Kamailio	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Forked-daapd	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.
Lighttpd1	10/10; 0	10/10; 0	10/10; 0	No planned-row shortfall is visible in the selected primary cache.

for example, uses the same target revision, instrumentation mode, coverage-replay procedure, target binary, and reset hook for the AFLNet-lineage systems. The `edges` field is the archived IPSM edge count and is used as a state-exploration diagnostic for AFLNet, ChatAFL, and LoopFuzz. For NSFuzz, the branch and edge fields are labeled as artifact counts unless the corresponding build and instrumentation procedure can be audited. The NSFuzz edge field is not semantically identical to the response-derived IPSM transition abstraction maintained by the AFLNet-lineage fuzzers, so the results separate branch coverage from state-exploration diagnostics.

For each primary target and metric, we summarize selected endpoint values with mean and sample SD. We also compute pairwise two-sided Mann-Whitney U tests comparing LoopFuzz against AFLNet and ChatAFL, followed by Benjamini-Hochberg FDR control across the 36 primary pairwise tests (nine targets, two metrics, and two baselines). Cliff's δ reports effect size, with positive values favoring LoopFuzz. We use the per-run terminal vectors in the archived selected summaries, after applying the same deterministic selection rule as Table 5. Exact tests are not always available because several rows contain ties, so the implementation uses SciPy's tied-rank handling. Crash discovery is outside this endpoint analysis because crash triage and root-cause validation are not normalized across the suite.

5.3. Policy-Ablation Design

The ablation design follows the exposed policy switches in the provided ablation archive: disabling counterexample-guided local repair, disabling frontier scheduling, disabling hypothesis validation, and disabling adaptive triggering. It does not include a full no-admission variant. A *LoopFuzz-no-admission* design would preserve the same model calls, prompts, query budget, plateau triggers, and scheduling policy, but bypass the $P/U/R/G$ admission checks for request-producing model suggestions. Table 10 therefore measures sensitivity to the integrated controller's exposed sub-policies rather than isolating the complete admission boundary. We apply the same deterministic run-selection rule as the primary comparison: keep completed runs with at least 1400 minutes, and if more than 10 valid runs remain, keep the lowest 10 run IDs. Because this archive is separate from `ten_groups_data_ten`, its full-controller column is not expected to match every primary-table endpoint exactly. All non-MQTT ablation cells reported here contain 10 eligible runs under this rule.

5.4. Trace Accounting and Cost Reporting

Because LoopFuzz is a closed-loop LLM-in-the-loop system, terminal coverage alone is not enough for mechanism accounting. The selected LoopFuzz archives preserve run-level fuzzer statistics and raw plateau-reply records. From those records we can recover model-call counts, plateau-call counts, token counters, prompt-reuse hits, and sampled hypothesis-validation counters. A complete candidate-level archive would additionally preserve, for every model proposal, the extracted candidate, $P/U/R/G$ decision, rejection reason, bounded-trial response, coverage/IPSM gain, local-repair event, and latency.

Table 4 reports the run-level mechanism counters recoverable from the selected primary LoopFuzz artifact archives. These values are not full admission rates: the archive does not contain normalized candidate-level admission-event logs, so $U/R/G$ pass rates, rejection-reason distributions, and per-candidate local-repair latency remain not recoverable. We therefore report token volume and validation counters rather than a cost-efficiency claim or a quantified semantic-drift prevalence rate.

Prompt-reuse deduplication hits were 0.0 ± 0.0 in the selected primary LoopFuzz cells, indicating that the 64-entry hash guard rarely encountered identical plateau contexts in these runs. Token counts are reported as volume counters only; the endpoint is provider-compatible and the archive does not define a stable price schedule, so we do not report dollar costs or cost-efficiency rankings.

6. Results

Table 5 summarizes the terminal aggregates used for the primary AFLNet and ChatAFL lineage comparison from `ten_groups_data_ten`. Figs. 3 and 4 visualize the same selected runs as per-target trajectories. This layout follows the protocol-level comparison view: each panel keeps one target on its own scale, so the curves show both endpoint

Table 4: Recoverable LoopFuzz mechanism counters from the selected primary artifact archives (mean ± sample SD over 10 selected runs). Calls, plateau calls, token counts, and sampled hypothesis-validation counters come from run-level fuzzer statistics; raw plateau replies are preserved as reviewer-auditable model-interaction records. These are run-level counters, not candidate-level $P/U/R/G$ admission rates.

Target	LLM calls	Plateau calls	Prompt tok. (k)	Compl. tok. (k)	Hyp. checks	Hyp. pass	Hyp. fitness
LightFTP	151.1±7.6	64.0±0.0	164.0±191.1	15.5±3.3	1162.1±177.9	0.0±0.0%	0.08±0.01
bftpd	246.6±3.9	64.0±0.0	154.4±11.3	24.9±1.6	891.5±115.9	5.4±12.0%	0.17±0.21
ProFTPD	196.7±9.1	14.1±3.5	86.5±3.9	17.1±0.9	436.8±111.0	2.7±5.2%	0.11±0.10
Pure-FTPd	228.2±6.2	64.0±0.0	474.2±550.9	21.9±1.4	1293.0±239.6	8.4±13.6%	0.16±0.17
Exim	105.4±13.1	15.5±5.9	49.5±16.4	9.1±2.6	266.1±120.8	0.0±0.0%	0.10±0.02
Live555	301.1±66.7	63.5±1.6	210.5±40.0	171.4±45.3	754.2±675.4	0.0±0.0%	0.18±0.17
Kamailio	51.3±23.9	17.5±8.8	63.2±39.0	51.1±29.3	3928.4±1963.2	0.0±0.0%	0.10±0.02
Forked-daapd	59.9±35.7	24.4±11.4	86.4±77.5	74.7±72.8	1722.8±144.5	0.0±0.0%	0.14±0.02
Lighttpd1	165.2±25.8	60.7±6.5	148.5±29.9	34.4±9.4	2715.3±1805.1	0.0±0.0%	0.12±0.03

separation and the time at which each fuzzer saturates. Table 6 reports the corrected rank-test summary. Together, the table and curves compare the integrated LoopFuzz strategy with its nearest AFLNet and ChatAFL predecessor chain.

The results below report endpoint outcomes. Table 4 separately records the recoverable run-level model-call, plateau-call, token, and sampled hypothesis-validation counters; candidate-level $P/U/R/G$ rates and rejection distributions remain unavailable in the current archive.

6.1. RQ1: Whole-Campaign Effect

Table 5 and Fig. 3 show target-dependent endpoints for direct open-loop assistance. ChatAFL ties AFLNet on LightFTP, finishes below AFLNet on Exim in `b_abs`, is close to AFLNet on Forked-daapd, and improves several other targets. The primary comparison therefore separates fluent generation from runtime-admitted campaign improvement.

Within the primary AFLNet/ChatAFL lineage, branch-coverage outcomes favor LoopFuzz on most targets but not uniformly. AFLNet remains highest on Forked-daapd, where LoopFuzz also has much larger variance. After BH correction, LoopFuzz differs from AFLNet on `b_abs` for eight targets, favoring LoopFuzz on seven and AFLNet on Forked-daapd. Against ChatAFL, corrected-significant differences appear on five targets, favoring LoopFuzz on four and ChatAFL on Forked-daapd. The remaining positive mean separations over ChatAFL, such as bftpd, ProFTPD, and Kamailio, are descriptive at the selected FDR threshold. Section 6.5 analyzes Forked-daapd directly.

Table 7 reports the auxiliary NSFuzz branch artifacts. LightFTP reports 414.3±9.8 NSFuzz artifact branches, while same-build AFLNet-lineage rows cluster around 71. Forked-daapd is also diagnostic: NSFuzz reports 2722.8±44.6, while AFLNet, ChatAFL, and LoopFuzz report 2326.3±77.3, 2315.3±50.1, and 2171.9±226.0.

6.2. RQ2: IPSM Escape Dynamics

Terminal `edges` provide the clearest primary-lineage evidence for improved state exploration. In the AFLNet/ChatAFL comparison, LoopFuzz has higher mean final `edges` than both lineage baselines on the nine primary targets, as visualized in Fig. 4. After BH correction, the LoopFuzz-versus-AFLNet edge advantage remains significant on eight targets; Forked-daapd remains positive by mean but not significant. Against ChatAFL, the corrected edge advantage remains significant on six targets, while Kamailio, Forked-daapd, and Lighttpd1 are positive by mean but not corrected-significant.

Table 8 keeps the NSFuzz edge archive separate from the AFLNet-lineage state-exploration metric. AFLNet, ChatAFL, and LoopFuzz report response-derived IPSM edges, whereas the NSFuzz column is an artifact edge count from a different state-aware implementation. The diagnostic mismatch is visible in the data: on LightFTP, NSFuzz has a much larger branch-count artifact in Table 7 but an artifact edge count of only 11.0±0.0; on Forked-daapd, NSFuzz reports 27.0±6.1 artifact edges while the AFLNet-lineage edge counts are 21.1±1.3, 20.5±2.1, and 22.1±1.5.

6.3. RQ3: End-of-Campaign Outcome

Endpoint outcomes separate by metric. In the primary three-way comparison, LoopFuzz is strongest by mean on most targets but does not lead Forked-daapd. The NSFuzz branch-count artifacts exceed LoopFuzz's AFLNet-lineage `b_abs` on five of eight nominal shared targets (LightFTP, Pure-FTPd, Exim, Kamailio, and Forked-daapd) and are lower on three (bftpd, ProFTPD, and Live555). Table 9 shows a different Mosquitto surface: MBFuzzer has a recoverable coverage-replay branch count, while LoopFuzz Mosquitto rows are much shorter than 24 hours and AFLNet-lineage `edges` are unavailable for MBFuzzer. Taken together, the archived endpoint evidence supports improved response-derived IPSM exploration within the AFLNet/ChatAFL lineage and a branch-coverage advantage on most primary targets, with mixed branch-count evidence from specialized external artifacts.

6.4. Auxiliary LightFTP Branch Surface

LightFTP has the largest gap between the primary-lineage `b_abs` values and the auxiliary NSFuzz branch-count artifact. In the primary table, all three AFLNet-lineage systems use the same LightFTP target revision,

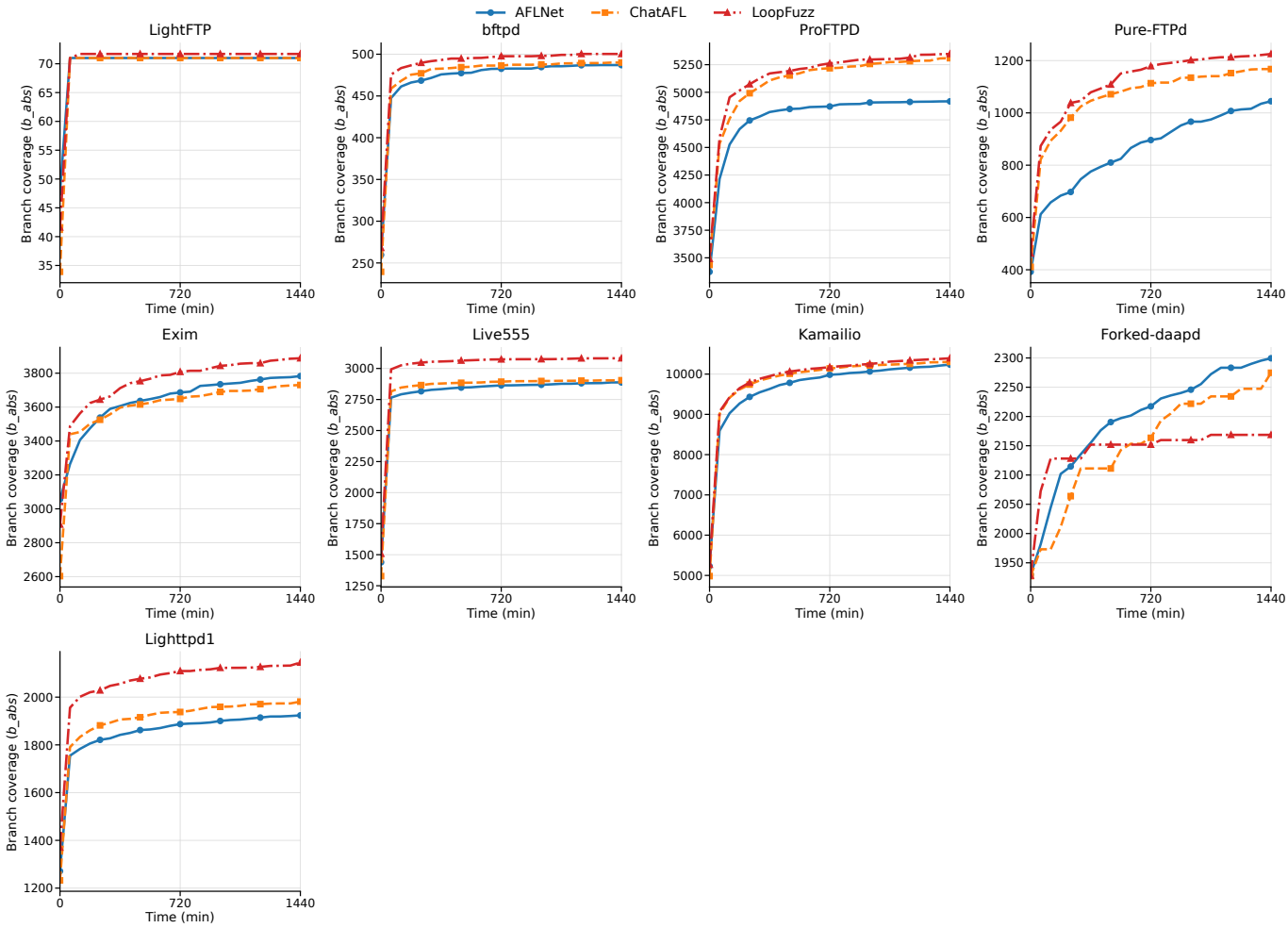

Figure 3: Branch-coverage trajectories for the nine primary-result targets. Each panel uses the same selected runs as Table 5.

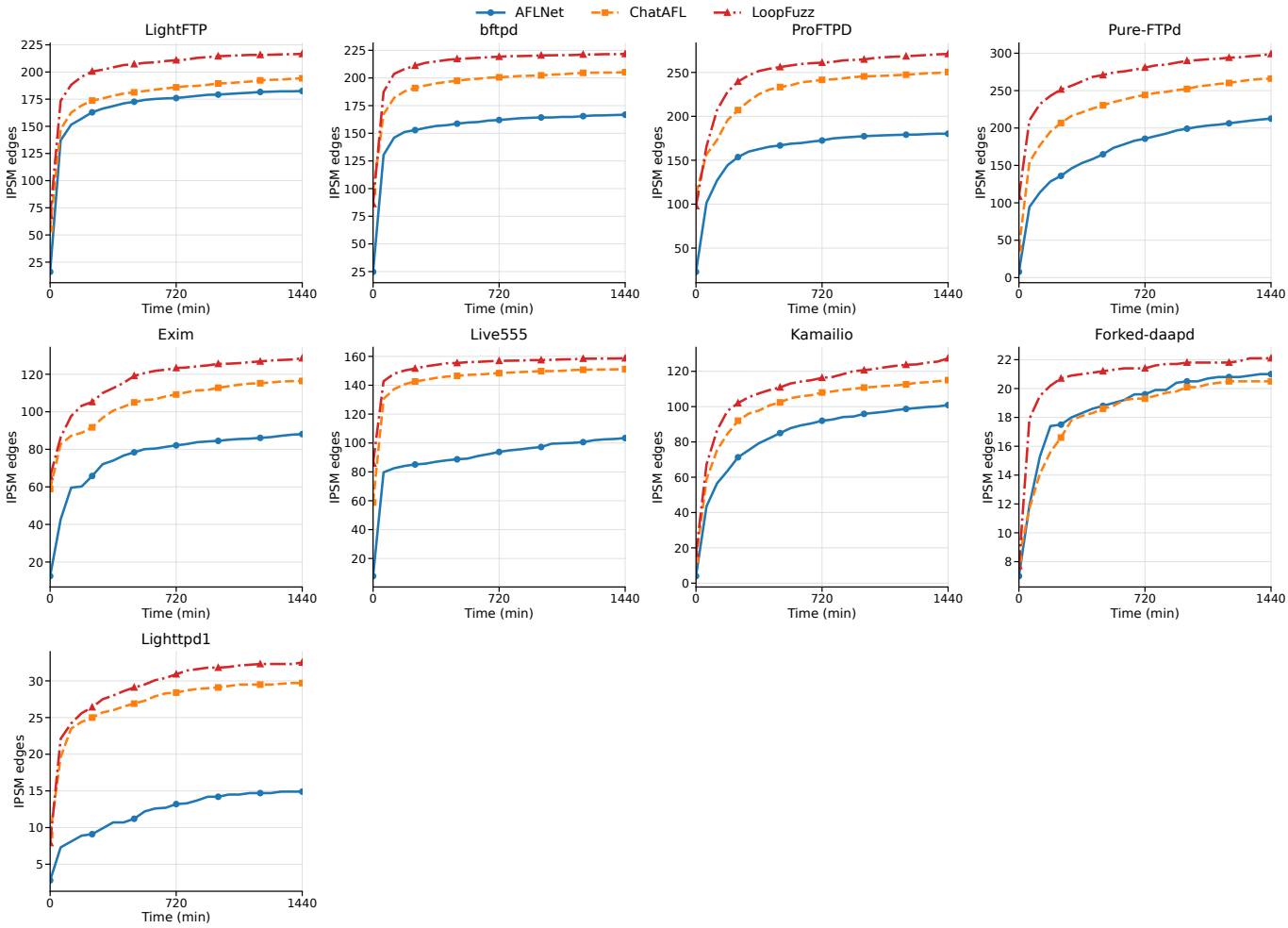

Figure 4: IPSM-edge exploration trajectories for the nine primary-result targets. Each panel uses the same selected runs as Table 5.

Table 5: Primary end-of-campaign results for `b_abs` and `edges` from the `ten_groups_data_ten` archive (mean ± sample SD over 10 selected completed runs per target-fuzzer cell). Bold marks the highest mean among AFLNet, ChatAFL, and LoopFuzz. Pairwise Mann-Whitney/BH/Cliff statistics for LoopFuzz versus the two baselines are summarized in Table 6.

Proto.	Target	Final `b_abs`			Final `edges`		
		AFLNet	ChatAFL	LoopFuzz	AFLNet	ChatAFL	LoopFuzz
FTP	LightFTP	71.0±0.0	71.0±0.0	**71.7±1.6**	182.8±10.2	194.2±10.2	**216.6±10.4**
FTP	bftpd	488.6±13.9	492.3±15.1	**501.5±7.4**	166.9±9.4	205.2±9.7	**221.7±16.9**
FTP	ProFTPD	4919.9±97.8	5317.3±101.7	**5358.8±136.1**	180.5±23.1	250.5±9.1	**271.7±18.8**
FTP	Pure-FTPd	1054.3±76.4	1179.2±31.0	**1240.5±25.8**	213.9±11.9	266.2±14.3	**299.6±26.2**
SMTP	Exim	3794.5±42.4	3746.2±47.7	**3908.5±43.2**	88.3±5.3	116.4±10.2	**128.5±7.8**
RTSP	Live555	2894.1±23.3	2911.1±10.9	**3088.7±22.4**	103.9±10.1	151.2±3.9	**158.7±8.5**
SIP	Kamailio	10244.0±118.1	10314.7±98.4	**10402.7±141.3**	101.1±7.7	115.7±13.9	**127.6±9.6**
DAAP/DACP	Forked-daapd	**2326.3±77.3**	2315.3±50.1	2171.9±226.0	21.1±1.3	20.5±2.1	**22.1±1.5**
HTTP	Lighttpd1	1930.4±18.7	1987.5±38.3	**2156.5±111.8**	14.9±3.8	29.7±4.7	**32.5±10.8**

Table 6: Pairwise statistical summary for LoopFuzz against each primary baseline. Each cell reports raw p / BH-adjusted q / Cliff's δ for the two-sided Mann-Whitney U test on selected per-run endpoints; positive δ favors LoopFuzz and negative values favor the baseline.

Target	`b_abs`		`edges`	
	vs. AFLNet	vs. ChatAFL	vs. AFLNet	vs. ChatAFL
LightFTP	0.078/0.093/ + 0.30	0.078/0.093/ + 0.30	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 0.94
bftpd	0.017/0.030/ + 0.64	0.150/0.169/ + 0.39	< 0.001/ < 0.001/ + 1.00	0.031/0.043/ + 0.58
ProFTPD	< 0.001/ < 0.001/ + 1.00	0.623/0.623/ + 0.14	< 0.001/ < 0.001/ + 1.00	0.012/0.022/ + 0.67
Pure-FTPd	< 0.001/ < 0.001/ + 1.00	< 0.001/0.002/ + 0.90	< 0.001/ < 0.001/ + 1.00	0.005/0.009/ + 0.76
Exim	< 0.001/0.002/ + 0.90	< 0.001/ < 0.001/ + 0.98	< 0.001/ < 0.001/ + 1.00	0.025/0.037/ + 0.60
Live555	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 1.00	0.034/0.045/ + 0.57
Kamailio	0.021/0.035/ + 0.62	0.345/0.365/ + 0.26	< 0.001/ < 0.001/ + 0.97	0.049/0.063/ + 0.53
Forked-daapd	0.026/0.037/ − 0.60	0.026/0.037/ − 0.60	0.164/0.179/ + 0.37	0.113/0.131/ + 0.42
Lighttpd1	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 0.95	< 0.001/ < 0.001/ + 0.98	0.596/0.613/ + 0.15

Table 7: Auxiliary NSFuzz branch-count artifacts. AFLNet, ChatAFL, and LoopFuzz same-build `b_abs` values appear in Table 5; the NSFuzz counts are reported separately because they come from a different artifact surface.

Target	NSFuzz artifact branch count
LightFTP	414.3±9.8
bftpd	477.6±3.9
ProFTPD	5335.0±86.9
Pure-FTPd	1332.2±8.8
Exim	4191.5±62.0
Live555	3021.5±42.1
Kamailio	10635.3±123.8
Forked-daapd	2722.8±44.6

Table 8: State-exploration diagnostics in the auxiliary archive. AFLNet, ChatAFL, and LoopFuzz report final response-derived AFLNet-lineage `edges`. The NSFuzz column reports *NSFuzz artifact edge count*. Bold marks the highest mean only among AFLNet-lineage columns.

Target	AFLNet `edges`	ChatAFL `edges`	LoopFuzz `edges`	NSFuzz artifact edge count
LightFTP	182.8±10.2	194.2±10.2	**216.6±10.4**	11.0±0.0
bftpd	166.9±9.4	205.2±9.7	**221.7±16.9**	48.7±4.8
ProFTPD	180.5±23.1	250.5±9.1	**271.7±18.8**	186.0±11.5
Pure-FTPd	213.9±11.9	266.2±14.3	**299.6±26.2**	17.6±0.5
Exim	88.3±5.3	116.4±10.2	**128.5±7.8**	13.9±1.2
Live555	103.9±10.1	151.2±3.9	**158.7±8.5**	49.8±2.0
Kamailio	101.1±7.7	115.7±13.9	**127.6±9.6**	14.3±0.7
Forked-daapd	21.1±1.3	20.5±2.1	**22.1±1.5**	27.0±6.1

fuzzing instrumentation, coverage instrumentation, replay procedure, target binary, and reset hook. Under that shared pipeline, the branch-coverage trajectory reaches 71.0 by 60 minutes for AFLNet and ChatAFL, and 71.7 by 120 minutes for LoopFuzz, then remains flat through 1440 minutes.

The NSFuzz artifact for the same nominal target is 414.3±9.8 branch-count units. The accompanying

Table 9: MQTT/Mosquitto artifact-integrity summary. The AFLNet row is a 24-hour AFLNet-lineage run, the LoopFuzz Mosquitto rows are much shorter, and MBFuzzer exposes coverage-replay branch counts from a Mosquitto-only worker archive with no AFLNet-lineage `edges`.

Artifact	Runs	Runtime (min)	Branch metric	Edge metric	Use in this paper
AFLNet Mosquitto	10	1479.0±0.0	`b_abs` 1814.3±147.3	`edges` 36.0±4.5	Same lineage but no ChatAFL row; excluded from the primary three-way table.
LoopFuzz Mosquitto	10	137.4±134.7	`b_abs` 2518.5±269.9	`edges` 107.7±18.1	Not same-budget with AFLNet; excluded from performance conclusions.
MBFuzzer Mosquitto	10	≈ 1480	replay branches 2827.0±58.9	N/A	External Mosquitto-only artifact; no primary-format `b_abs` or AFLNet-lineage `edges`.

NSFuzz artifact edge count is 11.0±0.0, while AFLNet, ChatAFL, and LoopFuzz report 182.8±10.2, 194.2±10.2, and 216.6±10.4 response-derived IPSM edges. This combination makes LightFTP the strongest example of cross-artifact metric divergence in the selected data.

LightFTP is therefore reported in Table 5 for the same-build AFLNet/ChatAFL/LoopFuzz lineage comparison and in Table 7 for the auxiliary NSFuzz branch-count surface.

6.5. Failure-Target Analysis: Forked-daapd

Forked-daapd is the clearest primary branch-coverage exception for LoopFuzz. In Table 5, LoopFuzz ends at 2171.9±226.0 final `b_abs`, below AFLNet at 2326.3±77.3 and ChatAFL at 2315.3±50.1. The variance is also diagnostic: LoopFuzz's branch-coverage standard deviation is about three times AFLNet's and more than four times ChatAFL's. Table 7 also reports a high NSFuzz artifact branch count of 2722.8±44.6 for this target. The same target separates state exploration from branch coverage: LoopFuzz improves response-derived IPSM exploration while missing branch coverage reached by the AFLNet-lineage baselines.

The endpoint data point to a state-signal mismatch rather than a simple coverage win. Forked-daapd is the only primary target where LoopFuzz loses `b_abs` by mean and corrected significance, yet it still has the highest mean AFLNet-lineage `edges` (22.1±1.5 versus 21.1±1.3 for AFLNet and 20.5±2.1 for ChatAFL), although the edge difference is not significant after correction. Thus, larger response-derived state structure does not translate into deeper branch exploration on this target.

A plausible explanation is that DAAP/DACP exposes a noisy or sparse response signal for the AFLNet-lineage IPSM abstraction. Forked-daapd contains media-library and control-session behavior where requests can change visible responses without exercising branches that dominate `b_abs`. LoopFuzz may therefore promote candidates that look state-progressive but are not branch-productive. The high variance is also compatible with initialization or early-history instability.

The ablation row points away from a simple "one switch is too strict" explanation. On Forked-daapd, removing local repair, frontier scheduling, hypothesis validation, or adaptive triggering all lowers `b_abs` relative to the full controller (-104.4, -145.0, -129.9, and -91.8). Some relaxed variants slightly increase `edges`, but not branch coverage. The target may need a different admission signal, such as richer DAAP/DACP response parsing, target-state normalization, or database/session initialization checks.

6.6. Ablation: Boundary-Condition Analysis

Table 10 reports the archived policy perturbations as deltas relative to the full-controller rows in the separate ablation archive. Each cell shows Δ `b_abs` / Δ `edges`. The perturbations measure component sensitivity within the integrated policy; the full-controller column is a local ablation baseline, not the selected primary-table LoopFuzz cell.

The clearest broad signal is frontier control: removing it reduces mean `b_abs` on eight of nine targets, with ProFTPD as the exception. This pattern suggests that state-directed budget allocation is broadly useful in the selected ablation archive. The remaining switches are more target-dependent.

Local repair is one non-monotone case. Removing local repair increases `b_abs` on ProFTPD by +15.8 and on Live555 by +1.2, while reducing `edges` on both targets (-16.3 and -1.5). This pattern is consistent with local repair sometimes over-constraining the message space: a repaired hypothesis can prevent repeated invalid continuations, but it can also narrow a target-specific region where looser mutations still reach additional instrumented branches. On the other seven targets, removing local repair reduces `b_abs`. ProFTPD and Live555 may need a less aggressive repair policy or a decay mechanism that reopens constraints after they stop producing coverage.

Adaptive triggering shows a different boundary condition. Removing it improves both `b_abs` and `edges` on Kamailio (+26.9/+5.2), and improves `b_abs` but not `edges` on Live555 (+5.0/-3.6). On Lighttpd1, removing it raises `edges` while lowering `b_abs`; on Pure-FTPd, both endpoints decline (-22.3/-4.6). These mixed deltas suggest that the plateau trigger can be too late or too conservative for some targets, while earlier or less gated intervention can also reduce endpoint quality.

Disabling hypothesis validation has the same non-monotone shape: `b_abs` drops on eight of nine targets, with Pure-FTPd as the exception (+4.6), while `edges` rises on Pure-FTPd, Kamailio, and Forked-daapd. This separates response-derived IPSM growth from branch endpoint quality. The ablation archive therefore supports a cautious interpretation: the full controller is broadly

stable, but individual sub-controls are target-dependent and are not uniformly necessary on every target.

7. Discussion

The paper makes one central design claim. In stateful protocol fuzzing, LLM-generated requests should be governed by runtime evidence before they change the durable queue. ChatAFL already showed that an LLM can generate protocol text [18]. LoopFuzz asks a different question: when should such output be trusted by the campaign? Our design answer is closed-loop runtime admission control. It checks structure and parseability before a request trial, then uses the trial response, state movement, and coverage gain to decide whether an LLM suggestion can strengthen the queue. Scheduling-only actions are weaker control hints rather than admitted inputs. The evaluated strategy combines request admission, local repair, and state-aware scheduling as one campaign-control policy.

The admission controller and the scheduler solve different parts of the same problem. Admission control filters low-value suggestions. It does not decide where to spend the recovered budget. State-aware scheduling decides where to search. Without closed-loop intervention, however, it still depends on ordinary mutation to cross semantic bottlenecks. LoopFuzz combines the two because admission control and state-directed search reinforce each other.

The results also clarify the role of response-derived state counts. A campaign can accumulate superficially different rejection paths without moving toward high-value logic. LoopFuzz therefore reads state exploration together with `b_abs` and demotes states that remain unproductive over repeated selections. In this setting, productive states matter more than raw state counts, so `edges` functions as a companion diagnostic for campaign behavior alongside absolute branch coverage.

Forked-daapd illustrates this state-signal mismatch. LoopFuzz grows the response-derived IPSM faster and ends with slightly more primary-lineage `edges`, but it underperforms AFLNet and ChatAFL in final branch coverage; the NSFuzz artifact branch count is also much higher. This outcome suggests that DAAP/DACP response-level progress can be too weak or noisy for the current admission and scheduling policy. Richer response parsing, target-state normalization, and database/session initialization checks are promising directions for this target family.

Counterexample-guided local repair is also a deliberate design choice. Repeated unconstrained regeneration does not accumulate persistent knowledge. Local repair turns failed executions into stored evidence. It makes runtime evidence cumulative rather than disposable. The ablation data show that this repair pressure is target-dependent: on ProFTPD and Live555, disabling local repair raises `b_abs` while lowering `edges`, which suggests that local repairs may sometimes over-shrink a search region that still contains branch-productive variants. Adaptive triggering has a related pattern: disabling it improves both reported endpoints on Kamailio and improves `b_abs` but not `edges` on Live555, while Pure-FTPd moves in the opposite direction. The complete strategy is therefore best characterized as a broadly stable integrated controller with target-dependent sub-controls, not as a proof that every module is independently optimal for every target.

8. Threats to Validity

Internal validity: LLM-assisted fuzzing remains sensitive to generation variance, target nondeterminism, and timing effects. We mitigate these threats by fixing the environment, pinning the model endpoint, fixing call-class sampling settings, bounding retries and timeouts, and using repeated 24-hour runs. The low-temperature paths are hypothesis generation and local repair (T=0.3), while plateau assistance deliberately uses a higher temperature (T=1.2) to propose exploratory scheduling actions. No model-side random seed is set, so repeated plateau prompts may still yield different replies even when the fuzzer state is similar. Each archived run is isolated in a container with a one-CPU quota and separate network namespace. However, the artifact lacks the global co-scheduling plan, host-load trace, container-utilization trace, CPU-frequency trace, and per-run PRNG seeds. It therefore cannot rule out residual wall-clock effects from contention, storage pressure, network jitter, scheduler decisions, or provider-side model nondeterminism. Growth-rate plots are descriptive wall-clock traces, not CPU-time-normalized throughput claims. Plateau triggering and local repair are history-dependent, so the study cannot cleanly separate model variance from control-policy variance.

Statistical conclusion validity: The primary table spans nine targets, two endpoint metrics, and two LoopFuzz-versus-baseline comparisons per metric. Table 6 reports raw Mann-Whitney p values, BH-adjusted q values, and Cliff's δ for all 36 tests from the selected per-run endpoint vectors. These tests are still limited by small sample size ($n = 10$ per cell), ties in some endpoint distributions, and the fact that the deterministic run-selection rule is based on recoverable archive rows rather than a complete launch ledger. The ablation table is computed from a separate repeated-run archive and is descriptive; its local full-controller rows are not pooled with the primary table. We therefore interpret corrected-significant rows as endpoint evidence within the archived selected runs, not as universal target behavior.

External validity: The benchmark suite spans 10 implementations across seven protocol families, but the primary three-way comparison covers the nine non-MQTT targets with comparable AFLNet, ChatAFL, and LoopFuzz rows. Evidence is strongest for textual or semi-textual protocols whose progress is visible in responses and for systems sharing the AFLNet/ChatAFL response-derived IPSM lineage. We do not claim unchanged transfer to opaque binary or encrypted protocols, nor superiority over all state-aware

Table 10: Policy-perturbation results from the separate `ten_groups_ablation_ten` archive. The first number in each cell is final `b_abs`; the second is final `edges`. The Full column reports the local full-controller mean final `b_abs` / `edges`; other cells report deltas relative to that local mean. These Full values are separate repeated-run cells and are not required to match the primary LoopFuzz values in Table 5. Runs with archived runtime below 1400 minutes are excluded; when more than 10 valid runs remain, we keep the lowest 10 run IDs. All non-MQTT ablation cells reported here contain 10 eligible runs under this rule.

Target	Full	No local repair	No frontier	No hypothesis	No adaptive trigger
LightFTP	71.6 / 213.9	-0.2 / -13.5	-0.6 / -14.2	-0.5 / -11.0	-0.6 / -1.3
bftpd	501.5 / 221.7	-3.1 / -7.5	-3.0 / -8.4	-5.2 / -5.0	-7.0 / -12.1
ProFTPD	5395.9 / 284.2	+15.8 / -16.3	+30.0 / -23.4	-69.6 / -22.1	-17.0 / -8.9
Pure-FTPd	1240.5 / 299.6	-23.6 / -18.3	-18.5 / -5.3	+4.6 / +0.9	-22.3 / -4.6
Exim	3908.5 / 128.5	-29.0 / -2.8	-39.7 / -2.9	-46.5 / -4.6	-27.7 / -4.2
Live555	3088.7 / 158.7	+1.2 / -1.5	-11.1 / -6.9	-12.2 / -4.4	+5.0 / -3.6
Kamailio	10402.7 / 127.6	-96.7 / +2.5	-121.4 / -5.0	-100.6 / +1.9	+26.9 / +5.2
Forked-daapd	2171.9 / 22.1	-104.4 / -0.9	-145.0 / +0.1	-129.9 / +0.7	-91.8 / -1.7
Lighttpd1	2156.5 / 32.5	-62.4 / -3.3	-15.9 / +1.9	-33.9 / -2.4	-27.1 / +4.1

fuzzers. The archive lacks same-format StateAFL, Bleem, and Logos reproductions under the same selection rule. NSFuzz artifact rows are target-specific diagnostics rather than a unified cross-system state-machine comparison, and MBFuzzer is available only as a Mosquitto worker archive without AFLNet-lineage `edges`.

Construct validity: We use final `b_abs` and final `edges` as proxies for fuzzing effectiveness, but neither fully captures semantic protocol progress. AFLNet-lineage `b_abs` rows use shared target and replay pipelines, while NSFuzz branch counts remain artifact counts unless their build, instrumentation, and coverage extraction can be audited; the LightFTP 414.3±9.8 artifact shows why this distinction matters. Runtime admission means structured output checks, bounded validation executions, response-grounded acceptability, state-reachability signals, and counterexample-guided local repair, not formal verification or a perfect pre-execution oracle. The ablation archive also lacks *LoopFuzz-no-admission*, and event-level admission logs are unavailable. Semantic drift is therefore specified operationally and illustrated by a trace, not quantified as a cross-target prevalence or acceptance rate. The endpoint analysis reports campaign-control and exploration metrics rather than vulnerability yield; crash deduplication, exploitability review, and cross-target triage remain outside the current uniform protocol.

Reproducibility validity: ChatAFL and LoopFuzz share the same pinned model endpoint, each primary target is drawn from one declared archive, and the paper states the deterministic run-selection rule. The artifact material documents container isolation, CPU and memory quotas, internal endpoints, timeout handling, result manifests, recovery logic, and the model-call configuration: temperature by call class, `max_tokens`=4096, retry counts, API timeouts, fixed system prompts, and failure handling. The prompt reuse filter improves operational reproducibility by skipping repeated plateau-context hashes, but it is not a cache of deterministic model outputs because plateau calls use T=1.2. The selected LoopFuzz artifact archives preserve run-level fuzzer statistics and raw plateau replies for auditing stochastic calls, but they still lack normalized candidate-level admission decisions, rejection reasons, repair-event latencies, and full pre-filter launch metadata. Table 3 reports the recoverable selected inventory, but the artifact lacks full pre-filter inventories and failure metadata for any runs not represented in the selected cache. Mosquitto/MQTT rows are excluded from performance conclusions because they lack a homogeneous comparison surface: LoopFuzz averages 137.4 minutes, AFLNet and MBFuzzer are close to 1480 minutes, and MBFuzzer exposes worker-level coverage-replay branch counts rather than AFLNet-lineage `b_abs`/`edges`. A complete package should also preserve fuzzer statistics, model-interaction records, raw prompts and replies, admission decisions, rejection reasons, repair events, latency, token counters, exclusion logs, launch manifests, batch schedules, resource traces, and PRNG seeds.

9. Conclusion

This evaluation supports a design lesson: fluent protocol text is insufficient for a stateful campaign. In the evaluated AFLNet and ChatAFL lineage, the integrated LoopFuzz control strategy combines runtime admission, local repair, and state-aware scheduling to reduce the risk of semantically premature model suggestions.

Under the primary AFLNet and ChatAFL lineage comparison, LoopFuzz's clearest effect is improved state exploration, with a selective branch-coverage advantage on the same benchmark archive. For response-derived IPSM `edges`, corrected tests favor LoopFuzz on eight targets versus AFLNet and six targets versus ChatAFL. For final `b_abs`, corrected tests favor LoopFuzz over AFLNet on seven targets and over ChatAFL on four targets, while Forked-daapd favors the baselines. This supports a lineage-specific state-exploration claim and a majority-target branch-coverage claim, not a universal coverage-dominance claim.

The broader baseline picture is mixed and incomplete. NSFuzz artifact counts are informative external diagnostics,

but they exceed LoopFuzz's AFLNet-lineage `b_abs` on five of eight nominal shared targets and are lower on three; the current package does not preserve enough NSFuzz-side build, instrumentation, replay, and coverage-extraction metadata to treat those rows as homogeneous `b_abs` rankings. MBFuzzer is recoverable only as a Mosquitto worker artifact with coverage-replay branch counts and no AFLNet-lineage `edges`. Forked-daapd is the clearest primary negative target: LoopFuzz ends with slightly more response-derived `edges`, but its branch coverage is below AFLNet and ChatAFL; the higher NSFuzz artifact count should be rerun under a common pipeline. Same-format StateAFL, Bleem, and Logos reproductions are not available in the current archive. The archived ablations show target-dependent sub-controls: local repair can over-constrain ProFTPD and Live555 branch exploration, frontier control has ProFTPD as an exception in the ablation archive, and hypothesis validation and adaptive triggering are not uniformly monotone. They also lack the *LoopFuzz-no-admission* condition needed to isolate the full admission boundary. The evidence therefore supports a precise conclusion: the integrated LoopFuzz control strategy improves state-exploration behavior within the closest AFLNet/ChatAFL lineage and improves absolute branch coverage on most primary targets, but this study does not establish the independent causal effect of the complete admission formula, per-module necessity on every target, or dominance over specialized external fuzzers.

CRediT authorship contribution statement

Author contribution roles will be confirmed by all authors before submission and transferred into the submission system or final manuscript.

Declaration of competing interest

The competing-interest declaration will be confirmed by all authors before submission.

Funding

Funding information will be confirmed by all authors before submission.

Data availability

Data and code availability statements will be finalized before submission. The submission package will include an anonymized GitHub artifact exposed to reviewers through `anonymous.4open.science`, together with a supplementary archive uploaded through the submission system. The planned package contains the core code, run scripts, target-version pins, container/build configuration material, protocol and fuzzer configuration material, statistical and table-generation scripts, prompt templates, raw selected and auxiliary summary tables, and representative logs needed to audit the tables in this paper. Credentials, local paths, and machine-specific identifiers will be removed from the review package. At the time of this draft, no public DOI or permanent repository URL has been assigned.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI Codex for language editing, LaTeX formatting, and submission-compliance checking. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] V.-T. Pham, M. B"ohme, A. Roychoudhury, AFLNet: A greybox fuzzer for network protocols, in: 2020 IEEE 13th International Conference on Software Testing, Verification and Validation (ICST), 2020, pp. 460–465. doi:10.1109/ICST46399.2020.00062.

[2] R. Natella, StateAFL: Greybox fuzzing for stateful network servers, Empirical Software Engineering 27 (2022). doi:10.1007/s10664-022-10233-3, art. no. 191.

[3] J. Ba, M. B"ohme, Z. Mirzamomen, A. Roychoudhury, Stateful greybox fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3255–3272.

[4] F. Hu, J. Ji, H. Shu, Z. Li, T. Liu, C. Zhang, Formatted stateful greybox fuzzing of TLS server, in: 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), 2024, pp. 151–160. doi:10.1109/icst60714.2024.00022.

[5] Q. Zhang, X. Bai, X. Li, H. Duan, Q. Li, Z. Li, Resolverfuzz: Automated discovery of DNS resolver vulnerabilities with query-response fuzzing, in: 33rd USENIX Security Symposium (USENIX Security 2024), 2024, pp. 4729–4746.

[6] X. Song, J. Wu, Y. Zeng, H. Pan, C. Zuo, Q. Zhao, S. Guo, MBFuzzer: A multi-party protocol fuzzer for MQTT brokers, in: 34th USENIX Security Symposium (USENIX Security 2025), 2025, pp. 6179–6197.

[7] Z. Luo, J. Yu, F. Zuo, J. Liu, Y. Jiang, T. Chen, A. Roychoudhury, J. Sun, Bleem: Packet sequence oriented fuzzing for protocol implementations, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 4481–4498.

[8] F. Wu, Z. Luo, Y. Zhao, Q. Du, J. Yu, R. Peng, H. Shi, Y. Jiang, Logos: Log guided fuzzing for protocol implementations, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024, pp. 1720–1732. doi:10.1145/3650212.3680394.

[9] N. Redini, A. Continella, D. Das, G. D. Pasquale, N. Spahn, A. Machiry, A. Bianchi, C. Kruegel, G. Vigna, DIANE: Identifying fuzzing triggers in apps to generate under-constrained inputs for iot devices, in: 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 484–500. doi:10.1109/SP40001.2021.00066.

[10] Y. Dong, S. M. M. Rashid, A. Ranjbar, T. Wu, S. R. Hussain, M. S. Mahmud, A. A. Ishtiaq, K. Tu, T. Yang, CoreCrisis: Threat-guided and context-aware iterative learning and fuzzing of 5g core networks, in: 34th USENIX Security Symposium (USENIX Security 2025), 2025, pp. 5287–5306.

[11] M. B"ohme, V.-T. Pham, M.-D. Nguyen, A. Roychoudhury, Directed greybox fuzzing, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2329–2344. doi:10.1145/3133956.3134020.

[12] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, A. L. Hosking, Seed selection for successful fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 230–243. doi:10.1145/3460319.3464829.

[13] M. Wu, L. Jiang, J. Xiang, Y. Huang, H. Cui, L. Zhang, Y. Zhang, One fuzzing strategy to rule them all, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1634–1645. doi:10.1145/3510003.3510174.

[14] X. Zhu, M. B"ohme, Regression greybox fuzzing, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 2169–2182. doi:10.1145/3460120.3484596.

[15] J. Ba, G. J. Duck, A. Roychoudhury, Efficient greybox fuzzing to detect memory errors, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–12. doi:10.1145/3551349.3561161.

[16] R. Natella, V.-T. Pham, ProFuzzBench: A benchmark for stateful protocol fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 662–665. doi:10.1145/3460319.3469077.

[17] A. Andronidis, C. Cadar, SnapFuzz: high-throughput fuzzing of network applications, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 340–351. doi:10.1145/3533767.3534376.

[18] R. Meng, M. Mirchev, M. B"ohme, A. Roychoudhury, Large language model guided protocol fuzzing, in: 31st Annual Network and Distributed System Security Symposium (NDSS 2024), 2024, pp. 1–17. doi:10.14722/ndss.2024.24556.

[19] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, L. Zhang, Fuzz4All: Universal fuzzing with large language models, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13. doi:10.1145/3597503.3639121.

[20] J. Wang, L. Yu, X. Luo, LLMIF: Augmented large language model for fuzzing IoT devices, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 881–896. doi:10.1109/sp54263.2024.00211.

[21] X. Ma, L. Luo, Q. Zeng, From one thousand pages of specification to unveiling hidden bugs: Large language model assisted fuzzing of matter IoT devices, in: 33rd USENIX Security Symposium (USENIX Security 2024), 2024, pp. 4783–4800.

[22] D. Wang, Y. Li, Z. Zhang, K. Chen, CarpetFuzz: Automatic program option constraint extraction from documentation for fuzzing, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 1919–1936.

[23] Y. Lyu, Y. Xie, P. Chen, H. Chen, Prompt fuzzing for fuzz driver generation, in: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 3793–3807. doi:10.1145/3658644.3670396.

[24] Y. Deng, C. S. Xia, H. Peng, C. Yang, L. Zhang, Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023, pp. 423–435. doi:10.1145/3597926.3598067.

[25] Y. Deng, C. S. Xia, C. Yang, S. Zhang, S. Yang, L. Zhang, Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 70:1–70:13. doi:10.1145/3597503.3623343.

[26] P. Srivastava, M. Payer, Gramatron: effective grammar-aware fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 244–256. doi:10.1145/3460319.3464814.

[27] J. Fu, J. Liang, Z. Wu, M. Wang, Y. Jiang, Griffin: Grammar-free dbms fuzzing, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–12. doi:10.1145/3551349.3560431.

[28] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, Y. Xiang, Snipuzz: Black-box fuzzing of IoT firmware via message snippet inference, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 337–350. doi:10.1145/3460120.3484543.

[29] C. Salls, C. Jindal, J. Corina, C. Kruegel, G. Vigna, Token-level fuzzing, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2795–2809.

[30] A. Bulekov, B. Das, S. Hajnoczi, M. Egele, No grammar, no problem: Towards fuzzing the Linux kernel without system-call descriptions, in: 30th Annual Network and Distributed System Security Symposium (NDSS 2023), 2023, pp. 1–16. doi:10.14722/ndss.2023.24688.

[31] A. Fioraldi, D. C. D'Elia, D. Balzarotti, The use of likely invariants as feedback for fuzzers, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2829–2846.

[32] C. Myung, G. Lee, B. Lee, MundoFuzz: Hypervisor fuzzing with statistical coverage testing and grammar inference, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 1257–1274.

[33] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, C. Zhang, NSFuzz: Towards efficient and state-aware network service fuzzing, ACM Transactions on Software Engineering and Methodology 32 (2023) 160:1–160:26. doi:10.1145/3580598.

[34] Y. Zhang, D. Fang, P. Liu, L. Xi, X. Lu, X. Chen, S. Si, L. Sun, MSGFuzzer: Message sequence guided industrial robot protocol fuzzing, in: 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), 2024, pp. 140–150. doi:10.1109/icst60714.2024.00021.

[35] K. Kim, T. Kim, E. Warraich, B. Lee, K. R. B. Butler, A. Bianchi, D. J. Tian, FuzzUSB: Hybrid stateful fuzzing of USB gadget stacks, in: 2022 IEEE Symposium on Security and Privacy (SP), 2022, pp. 2212–2229. doi:10.1109/sp46214.2022.9833593.

[36] P. Fiterau-Brostean, B. Jonsson, K. Sagonas, F. Taquist, DTLS-Fuzzer: A DTLS protocol state fuzzer, in: 2022 IEEE Conference on Software Testing, Verification and Validation (ICST), 2022, pp. 456–458. doi:10.1109/icst53961.2022.00051.

[37] K. Sagonas, T. Typaldos, EDHOC-Fuzzer: An EDHOC protocol state fuzzer, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023, pp. 1495–1498. doi:10.1145/3597926.3604922.

[38] Q. Zhang, Y. Chu, Y. Shen, J. Liu, H. Shi, Y. Jiang, W. Chang, Tron: Fuzzing Linux network stack via protocol-system call payload synthesis, in: 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025, pp. 3227–3238. doi:10.1109/ase63991.2025.00266.

[39] H. Kim, M. O. Ozmen, A. Bianchi, Z. B. Celik, D. Xu, PGFUZZ: Policy-guided fuzzing for robotic vehicles, in: 28th Annual Network and Distributed System Security Symposium (NDSS 2021), 2021, pp. 1–18. doi:10.14722/ndss.2021.24096.

[40] Y. Aafer, W. You, Y. Sun, Y. Shi, X. Zhang, H. Yin, Android smarttvs vulnerability discovery via log-guided fuzzing, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2759–2776.

[41] T. Scharnowski, N. Bars, M. Schloegel, E. Gustafson, M. Muench, G. Vigna, C. Kruegel, T. Holz, A. Abbasi, Fuzzware: Using precise MMIO modeling for effective firmware fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 1239–1256.

[42] B. Zhao, Z. Li, S. Qin, Z. Ma, M. Yuan, W. Zhu, Z. Tian, C. Zhang, StateFuzz: System call-based state-aware linux driver fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3273–3289.

[43] D. Ryu, G. Kim, D. Lee, S. Kim, S. Bae, J. Rhee, T. Kim, Differential fuzzing for data distribution service programs with dynamic configuration, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 807–818. doi:10.1145/3691620.3695073.

[44] T. Trippel, K. G. Shin, A. Chernyakhovsky, G. Kelly, D. Rizzo, M. Hicks, Fuzzing hardware like software, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3237–3254.

[45] M. Beckmann, J. Steffan, Coverage-guided fuzzing of embedded systems leveraging hardware tracing, in: Computer Security – ESORICS 2023, 2023, pp. 362–378. doi:10.1007/978-3-031-25460-4_21.

[46] Z. Du, Y. Li, Y. Liu, B. Mao, WindRanger: A directed greybox fuzzer driven by deviation basic blocks, in: Proceedings of the 44th International Conference on Software Engineering, 2022,

pp. 2440–2451. doi:10.1145/3510003.3510197.

[47] P. Lin, P. Wang, X. Zhou, W. Xie, X. Ren, K. Lu, When control flows deviate: Directed grey-box fuzzing with probabilistic reachability analysis, in: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering, 2025, pp. 713–725. doi:10.1109/ASE63991.2025.00065.

[48] Y. Chen, R. Zhong, H. Hu, H. Zhang, Y. Yang, D. Wu, W. Lee, One Engine to Fuzz 'em All: Generic language processor testing with semantic validation, in: 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 642–658. doi:10.1109/SP40001.2021.00071.

[49] C. Zhang, X. Lin, Y. Li, Y. Xue, J. Xie, H. Chen, X. Ying, J. Wang, Y. Liu, APICraft: Fuzz driver generation for closed-source sdk libraries, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2811–2828.

[50] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, H. Veith, Counterexample-guided abstraction refinement, in: Computer Aided Verification, volume 1855 of *Lecture Notes in Computer Science*, Springer, 2000, pp. 154–169. doi:10.1007/10722167_15.

[51] J. Jiang, H. Xu, Y. Zhou, RULF: Rust library fuzzing via api dependency graph traversal, in: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, 2021, pp. 581–592. doi:10.1109/ASE51524.2021.9678813.

[52] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, S. K. Cha, SMARTIAN: Enhancing smart contract fuzzing with static and dynamic data-flow analyses, in: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, 2021, pp. 227–239. doi:10.1109/ASE51524.2021.9678888.

[53] Z. Zhou, Y. Yang, S. Wu, Y. Huang, B. Chen, X. Peng, Magneto: A step-wise approach to exploit vulnerabilities in dependent libraries via llm-empowered directed fuzzing, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1633–1644. doi:10.1145/3691620.3695531.

[54] D. Xie, Y. Li, M. Kim, H. V. Pham, L. Tan, X. Zhang, M. W. Godfrey, DocTer: documentation-guided fuzzing for testing deep learning API functions, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 176–188. doi:10.1145/3533767.3534220.

[55] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench, T. Holz, SoK: Prudent evaluation practices for fuzzing, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 1974–1993. doi:10.1109/sp54263.2024.00137.

[56] M. B"ohme, L. Szekeres, J. Metzman, On the reliability of coverage-based fuzzer benchmarking, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1621–1633. doi:10.1145/3510003.3510230.

[57] C. Poncelet, K. K. Sagonas, N. Tsiftes, So many fuzzers, so little time: Experience from evaluating fuzzers on the contiking network stack, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–13. doi:10.1145/3551349.3556946.